ICT912 Programming

Topic 4
Repetition Control Structures

CIHE Australia

Resources

Prescribed Texts

- McMullen, K., Matthews, E., & Parsons, J. J. (2022). Programming with Python (1st ed.)., Cengage Learning

Recommended Readings

- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media, Inc

Lesson Learned from Week 3:

- Describe the purpose of control structures
- Identify and define relational operators
- Explain if-then, if-then-else, nested-if, else-if decision structures
  - Illustrate using flowcharts
- Identify logical operators in conditional statements

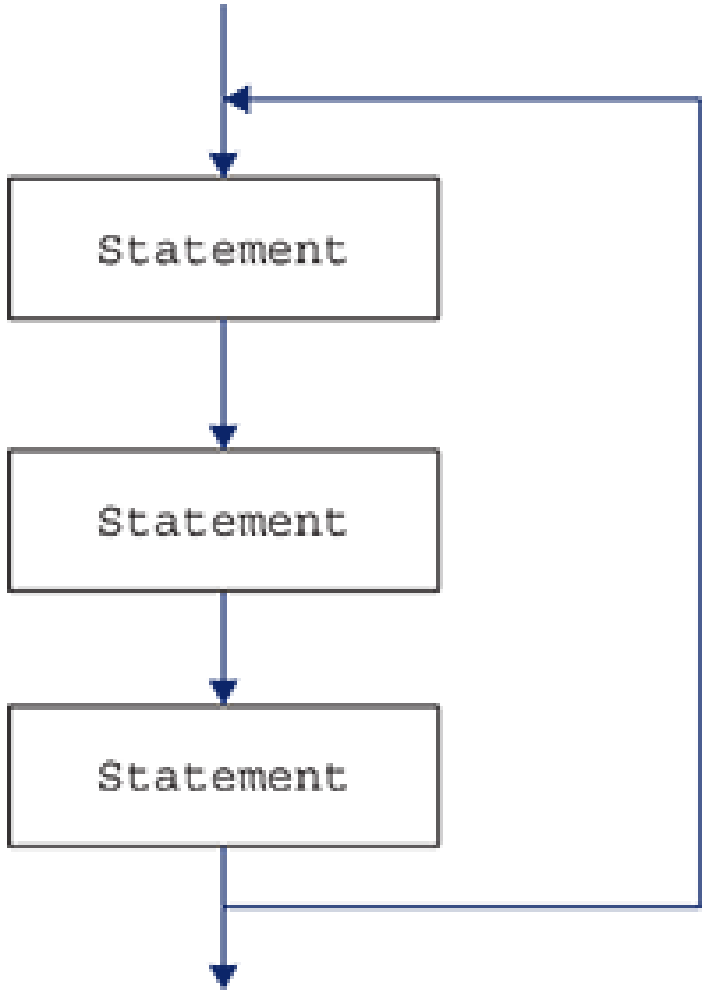
Week 4: Lesson Learning Outcomes

- Understand the need for the repetition control structure
- Explore various types of repetition control structures
  - for loop, nested loop, pre-test loops
- Advantages and disadvantages of the various structures

Loops Basics

- Computer programs can perform reps using **repetition control structures**, or **loops**
- Loops is a block of code with an entry point, a few statements that repeat, and an exit point

- Loops are handy for many tasks:
  - Processing series of records in a database
  - Keeping a running count
  - Accumulating totals
  - Solving recursive math problems



Types of Repetition Control Structures

- Count-controlled loops
  - Simplest repetition control structure which repeats a specified number of times.

- Pre-test loops
  - Condition-controlled loop which is regulated by conditional statements

Count-Controlled Loops

- Begins with a **control statement** containing the following parameters:
  - A variable for the **loop counter** that tracks the number of times a loop has repeated
  - An **iterative sequence** for the loop counter to use to control the loop

- Iterative sequence is data in Python that can be linearly traced (E.g. 1,2,3,4)
- The most frequently used iterative sequence uses the **range function**, which will create a numerical iterative sequence
  - based on starting number (optional), ending number, and incremental value (optional) parameters

range() Function

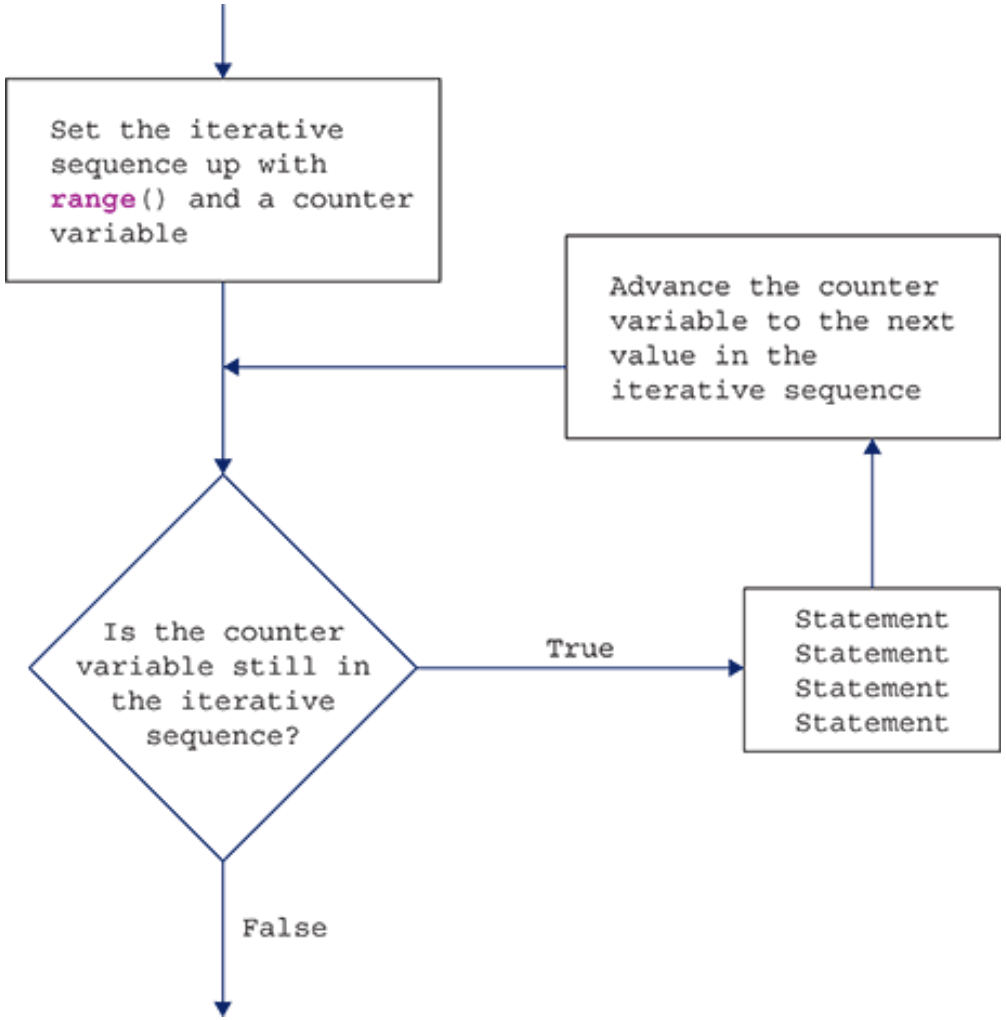
- Create a numerical iterative sequence based on its own parameters:
  - A starting number
  - An ending number
  - An incremental value, used to skip numbers in the sequence

Range Function Parameters	Iterative Sequence
range(10)	0 1 2 3 4 5 6 7 8 9
range(1, 10)	1 2 3 4 5 6 7 8 9
range(1, 10, 2)	1 3 5 7 9
range(0, 10, 2)	0 2 4 6 8

For loop

- A count-controlled loop that is appropriate for when you know how many times you want a loop to repeat (i.e. number of desired iterations)

- Iteration: a synonym for repetition



Contd…

Syntax:

```
for i in range(iteration):
    statement1
    statement2
```

Rules and best practices:

- The loop variable should be a simple identifier, usually `i`.
- The `range` function provides a sequence for `i` to iterate over.
- The loop variable will visit each value in the sequence provided by the range function, even if you change the loop variable inside the loop.
- Avoid modifying the loop counter variable within the loop. For example, do not assign to `i` inside the loop.

```
for i in range(1, 4):
    print("Lift")
    print("Pause")
    print("Lower")
    print()
```

OUTPUT:

```
Lift
Pause
Lower

Lift
Pause
Lower

Lift
Pause
Lower
```

User-Controlled Loops

- Allow the user to determine the number of times a loop repeats
- Specifies the value for range() function during runtime by obtaining user input for its value

```
reps = int(input("How many reps? "))
for i in range(reps):
    print("Lift, pause, lower")
```

OUTPUT:

```
How many reps? 5 [Enter]
Lift, pause, lower
Lift, pause, lower
Lift, pause, lower
Lift, pause, lower
Lift, pause, lower
```

Counters and Accumulators

- **Loops that count:**
  - Count-controlled loops have a built-in counter that you can use to count or accumulate totals

- The range() function can count by different increments when supplied with all three parameters

```
for i in range(2, 11, 2):
    print(i)
```

OUTPUT:

```
2
4
6
8
10
```

Notice that this sample loop uses a separate variable for the output and increments it within the loop to avoid using the loop counter variable within the loop

Contd…

- Loops that accumulate:
  - Accumulate a total by repeatedly adding values.
  - Syntax for accumulator spans two steps:
    - One step sets the accumulator to a starting value (0 for addition)
    - Other increases the accumulator by some amount

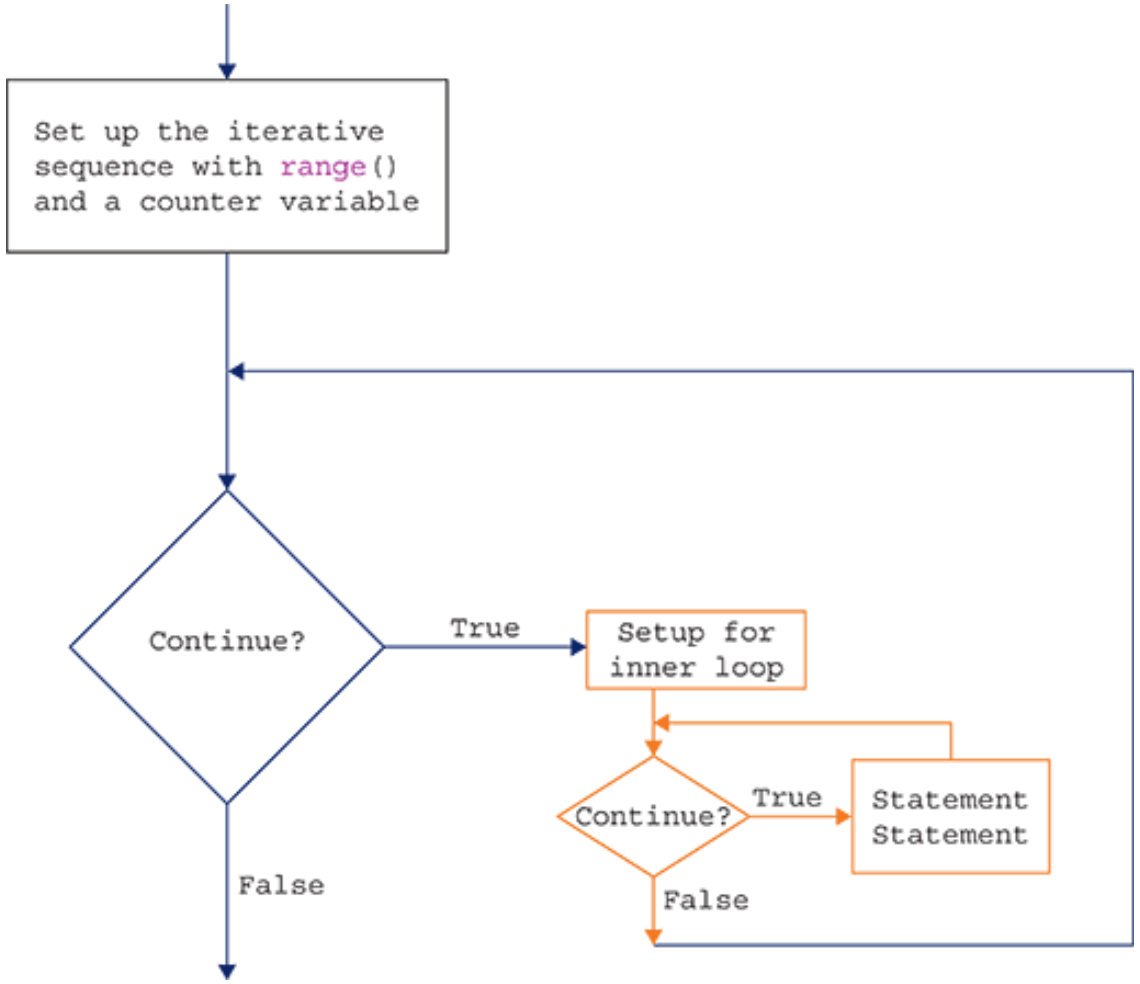
```python
# Accumulator example with loop

total = 0  # accumulator variable

# Repeat 3 times
for i in range(3):
    num = int(input("Enter a number: "))
    total = total + num  # add to accumulator
    print("The accumulator is:", total)

print("The total is:", total)
```

Output

```
Enter a number: 5
The accumulator is: 5
Enter a number: 7
The accumulator is: 12
Enter a number: 9
The accumulator is: 21
The total is: 21
```

Nested Loops

- Loops within loops:
  - Contain multiple control statements and loop counters



Inner and Outer Loops

- The inner loop is indented from the outer loop
- Each loop control variable must be unique

```
# Nested for loop example

for i in range(1, 4):  # outer loop
    for j in range(1, 3):  # inner loop
        print("i =", i, "j =", j)
```

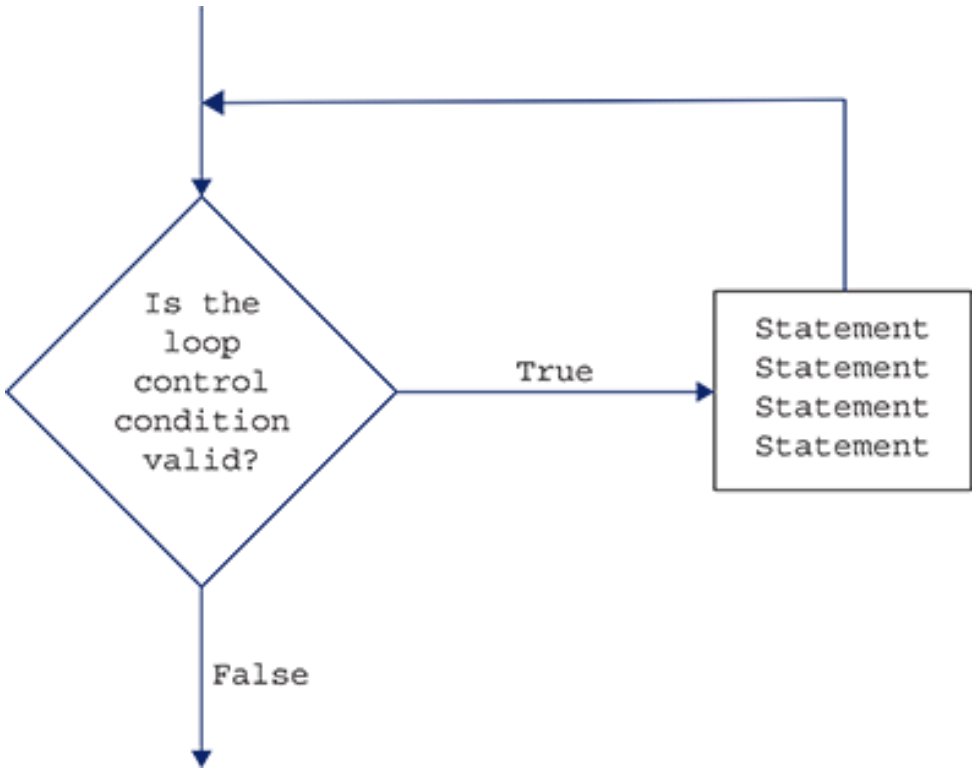
Output

```
i = 1 j = 1
i = 1 j = 2
i = 2 j = 1
i = 2 j = 2
i = 3 j = 1
i = 3 j = 2
```

Pre-Test Loops

- Condition-controlled loops are regulated by conditional statements containing Boolean expressions similar to those that control decision structures

- A pre-test loop begins with a condition
  - Loop executes only if the test condition is true
  - Continues executing until the condition is false or the loop encounters a `break` statement



while Loop

Syntax:

```
while condition:
    statement1
    statement2
```

- Loop that provides the syntax for implementing a pre-test loop

```
# While loop example

count = 1

while count <= 5:
    print("Count is:", count)
    count = count + 1
```

Output

```
Count is: 1
Count is: 2
Count is: 3
Count is: 4
Count is: 5
```

Infinite Loops

- A loop that continues forever
- Typical causes:
  - Invalid input that does not end an input loop
  - Input that changes the value of the loop counter
  - Programmer error in specifying the loop conditional statement
  - Using the loop counter variable in a mathematical operation inside the loop
  - Using the same loop counter variable for the inner and outer loops of a nested loop
- Ctrl+C may terminate an infinite loop
- Online compilers usually have a Stop button for use in such a situation

Example

```python
# Infinite loop example

while True:
    print("This will run forever!")
```

Output

```
This will run forever!
This will run forever!
This will run forever!
This will run forever!
This will run forever!
This will run forever!
This will run forever!
This will run forever!
This will run forever!
This will run forever!
This will run forever!
This will run forever!
This will run forever!
This will run forever!
This will run forever!
This will run forever!
This will run forever!
This will run forever!
This will run forever!
```

Breaking out of loops

- To prevent infinite loops that affect, provide a clean exit path from a loop preferably from within the logic of the loop control statement

- A break statement can be used to exit a loop, but the result might have unintended consequences
- Better practice is to design the control parameters to provide a graceful way to exit loops

```python
# Safe infinite loop with break

count = 1

while True:  # looks infinite
    print("This will not run forever.")
    count = count + 1

    if count > 5:  # exit condition
        break  # stops the loop
```

Output

```
This will not run forever.
This will not run forever.
This will not run forever.
This will not run forever.
This will not run forever.
```

Next Week – What's Coming Up?

- Understand the List as a composite data type
- Explore the types of lists
- Identify variables that represent a dictionary

CIHE

CIHE Australia